ECEN4223-5080 Spring 2026
Mobile Robotics
Project 1

Due date: Submit your report by 11:59 p.m. Monday, Feb. 16th, 2026 to Canvas.

I. Objectives

1. Get familiar with the installation of Ubuntu and ROS 2
2. Get familiar with the basic knowledge and programming in ROS 2

II. Tasks | Submission requirements are highlighted in bold and underlined text

1. Install Ubuntu 24.04 on your computer. After installation, validate your system version.
   For this task, please refer to **Appendix I**.
   <u>Submit your validation result</u>.

2. **<u>Explain the meaning of the following commands</u>:**
   2.1. sudo apt update
   2.2. sudo apt upgrade
   2.3. mkdir -p ~/ros2_ws/src
   2.4. cd ~/ros2_ws/src, cd .., cd ~
   2.5. ls
   2.6. pwd
   2.7. rm -ri ~/ros2_ws/src
   2.8. cp filename1 filename2, cp filename1 directory
   2.9. mv filename1 filename2, mv filename1 directory

3. Install ROS 2 Jazzy. After installation, validate the ROS version.
   For this task, please refer to **Appendix II**.
   <u>Submit your validation result</u>.

4. Install VS Code and connect it to WSL-Ubuntu. Then, connect to the WSL Ubuntu system through VS Code, and use the VS Code GUI to display your Ubuntu file system.
   For this task, please refer to **Appendix III**.
   <u>Submit a screenshot of the interface</u>.

5. Go to the ROS tutorial at https://docs.ros.org/en/jazzy/Tutorials.html.
   Read and work through the tutorials in the following order:
   - *Beginner: CLI tools* – complete all 10 tutorials in this section.

- *Beginner: Client libraries* – complete the first 5 tutorials only (stop after "Writing a simple publisher and subscriber (Python)").

Note: No submission requirement for this task.

6. Twist message generation and negation

In ROS, a Twist message represents motion using velocities. It contains two 3-D vectors:

- Linear velocity: motion along the x, y, z axes
- Angular velocity: rotation around the x, y, z axes

The Twist message is defined in the package geometry_msgs as: geometry_msgs/msg/Twist

In this task, you will create two ROS nodes that communicate using the publish/subscribe model. The nodes exchange data using Twist messages.

- **Node 1: Generator node**

  The generator node is responsible for creating and publishing random motion commands.

  <u>Requirements</u>

  a. Generate a Twist message with random values:
     - Linear velocity | x, y, z: random floating-point values in the range [-5, 5].
     - Angular velocity | x, y, z: random floating-point values in the range [-5, 5].

  b. Publish the Twist message:
     - Topic name: twist_data
     - Rate: 1 message per second (1 Hz)

  c. Print the generated Twist message to the terminal.

  d. Subscribe to the topic:
     - Topic name: negated_twist_data
     - Purpose: receive the negated Twist message from the Negator node
     - Print the received (negated) message to the terminal.

- **Node 2: Negator node**

  The negator node listens to the Generator and processes its messages.

  <u>Requirements</u>

  a. Subscribe to the topic:
     - Topic name: twist_data

  b. Negate the Twist message:
     - Multiply all linear and angular velocity components by -1.

c. Publish the negated Twist message:
   - Topic name: negated_twist_data

<u>Submit your code and screenshots of running results</u>.

Expected behavior

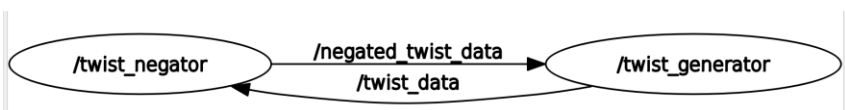
When both nodes are running:

- The Generator prints the original Twist.
- The Negator prints the negated Twist.
- The Generator prints the received negated Twist.
- Messages are exchanged once per second.

Output from the Generator node:

```
clau@LAPTOP-FLFKRVEF: ~/ros2_ws
clau@LAPTOP-FLFKRVEF:~/ros2_ws$ ros2 run project1 generator_node.py
[INFO] [1768421757.526227543] [twist_generator]: Generator started (Python). Publishing /twist_data at 1
 Hz.
[INFO] [1768421758.519928786] [twist_generator]: Publishing original Twist: linear[-1.52, -1.57, -1.19],
 angular[-3.04, -1.84, -3.78]
[INFO] [1768421759.519715987] [twist_generator]: Publishing original Twist: linear[-0.71, -4.60, -1.49],
 angular[1.56, 1.68, -0.80]
[INFO] [1768421759.521343993] [twist_generator]: Received negated Twist: linear[0.71, 4.60, 1.49], angul
ar[-1.56, -1.68, 0.80]
[INFO] [1768421760.519862090] [twist_generator]: Publishing original Twist: linear[-2.60, 4.59, -4.21],
angular[2.24, -4.46, 2.93]
[INFO] [1768421760.520837022] [twist_generator]: Received negated Twist: linear[2.60, -4.59, 4.21], angu
lar[-2.24, 4.46, -2.93]
[INFO] [1768421761.519939240] [twist_generator]: Publishing original Twist: linear[2.39, -3.48, -0.76],
angular[1.75, 3.87, -0.58]
[INFO] [1768421761.522604350] [twist_generator]: Received negated Twist: linear[-2.39, 3.48, 0.76], angu
lar[-1.75, -3.87, 0.58]
[INFO] [1768421762.519773155] [twist_generator]: Publishing original Twist: linear[2.64, 4.95, 3.47], an
gular[-2.42, 2.79, 2.64]
[INFO] [1768421762.520735999] [twist_generator]: Received negated Twist: linear[-2.64, -4.95, -3.47], an
gular[2.42, -2.79, -2.64]
[INFO] [1768421763.520317827] [twist_generator]: Publishing original Twist: linear[-1.99, -2.07, 4.51],
angular[-2.15, 1.92, 4.71]
[INFO] [1768421763.522376936] [twist_generator]: Received negated Twist: linear[1.99, 2.07, -4.51], angu
lar[2.15, -1.92, -4.71]
```

Communication graph:



Note: The following code is an example to show how to define a Twist message:

```python
from geometry_msgs.msg import Twist

msg = Twist()

msg.linear.x = 1.0
msg.linear.y = 1.0
msg.linear.z = 1.0

msg.angular.x = 2.0
msg.angular.y = 2.0
msg.angular.z = 2.0
```

7. Turtle controller

In this task, you will control a simulated turtle using ROS velocity commands. You will send motion commands to the turtle and observe its position and orientation in real time.

The turtle is simulated using the turtlesim package. Movement commands are sent using Twist messages, and the turtle's pose (position and orientation) is provided through a pose topic

Task overview

You will implement a node called *movement_controller* that controls the turtle's motion. The user selects a shape using the keyboard, and the turtle draws that shape exactly once, then stops.

Setup

- Start the turtlesim simulator:
  $: ros2 run turtlesim turtlesim_node
- View the turtle's pose in another terminal:
  $: ros2 topic echo /turtle1/pose

Node: *movement_controller*

Topics used:

Topic	Message Type	Direction	Purpose
/turtle1/cmd_vel	Geometry_msgs/msg/Twist	Publish	Send linear and angular velocity commands
/turtle1/pose	Turtlesim/msg/Pose	Subscribe	Read turtle position and orientation

User input

While the controller node is running, the user enters a command in another terminal:

Key	Action
C	Draw one circle
R	Draw one rectangle
T	Draw one triangle
Q	Quit the controller program

After completing the shape, the turtle must stop completely and wait for the next user command. If the user enters "Q", the controller must exit the program.

<u>Submit your code and screenshots of running results</u>.

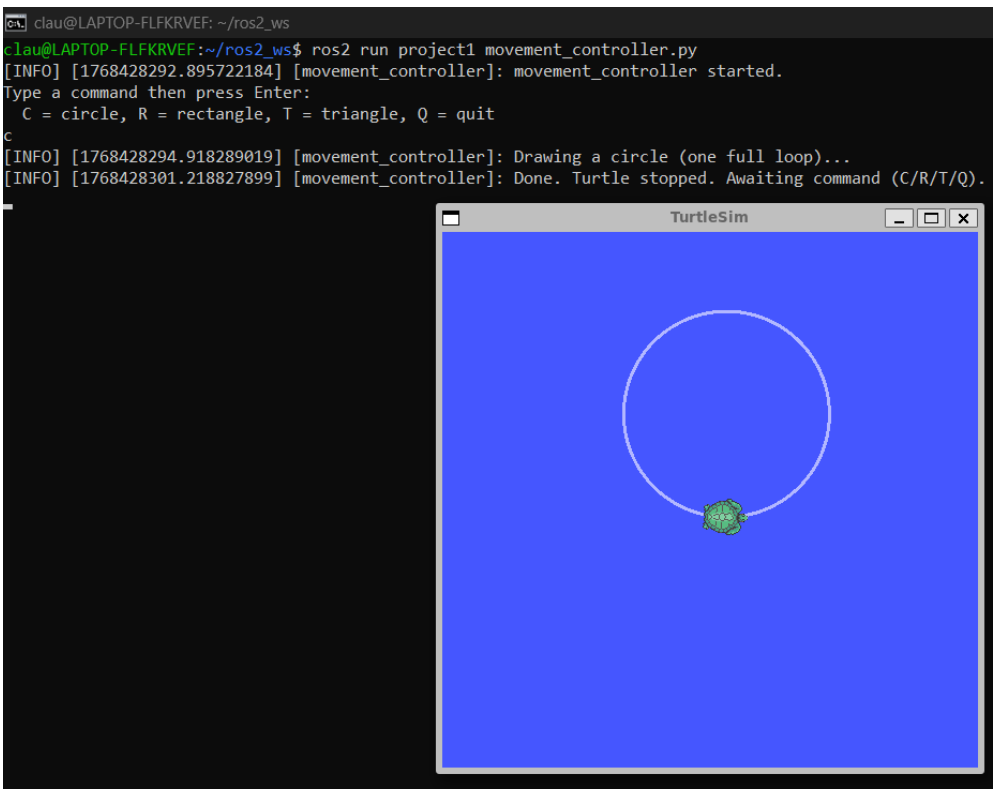
Note 1: You are free to choose the numerical values of the linear and angular velocities. Any values are acceptable as long as the turtle draws the correct shape, completes it exactly once, and stops afterward.

Expected behavior

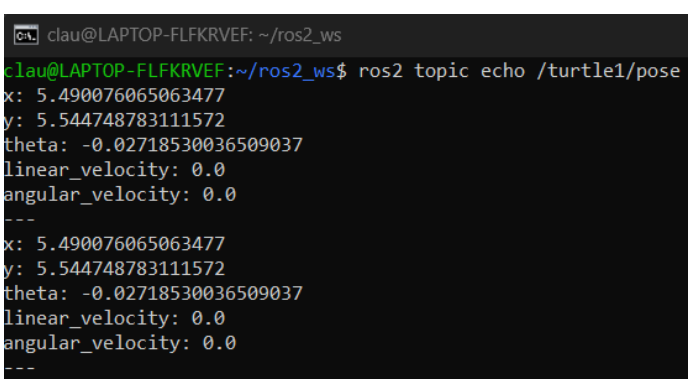
When the system is running:

- The turtle moves according to the selected shape.
- The turtle's pose (x, y, theta) updates in real time.
- The communication graph between nodes

Output from the *movement_controller* node and the simulator:



Output from the /turtle1/pose topic:



Appendix

Note: In this document, "**$:**" is the symbol of command, but not part of the command. For example, "**$: cd /my_folder**" means you just need to type "**cd /my_folder**" in the terminal.

I. Installing Ubuntu

Install Ubuntu on Windows 10/11 through WSL2

1. Open a PowerShell prompt as an Administrator.

2. Find the installable list of the Ubuntu system:
   $: wsl --list --online

```
(base) PS C:\WINDOWS\system32> wsl --list --online
The following is a list of valid distributions that can be installed.
Install using 'wsl.exe --install <Distro>'.

NAME                            FRIENDLY NAME
Ubuntu                          Ubuntu
Ubuntu-24.04                    Ubuntu 24.04 LTS
openSUSE-Tumbleweed             openSUSE Tumbleweed
openSUSE-Leap-16.0              openSUSE Leap 16.0
SUSE-Linux-Enterprise-15-SP7    SUSE Linux Enterprise 15 SP7
SUSE-Linux-Enterprise-16.0      SUSE Linux Enterprise 16.0
kali-linux                      Kali Linux Rolling
Debian                          Debian GNU/Linux
AlmaLinux-8                     AlmaLinux OS 8
AlmaLinux-9                     AlmaLinux OS 9
AlmaLinux-Kitten-10             AlmaLinux OS Kitten 10
AlmaLinux-10                    AlmaLinux OS 10
archlinux                       Arch Linux
FedoraLinux-43                  Fedora Linux 43
FedoraLinux-42                  Fedora Linux 42
eLxr                            eLxr 12.12.0.0 GNU/Linux
Ubuntu-20.04                    Ubuntu 20.04 LTS
Ubuntu-22.04                    Ubuntu 22.04 LTS
OracleLinux_7_9                 Oracle Linux 7.9
OracleLinux_8_10                Oracle Linux 8.10
OracleLinux_9_5                 Oracle Linux 9.5
openSUSE-Leap-15.6              openSUSE Leap 15.6
SUSE-Linux-Enterprise-15-SP6    SUSE Linux Enterprise 15 SP6
```

3. Install Ubuntu 22.04:
   $: wsl --install -d Ubuntu-24.04

4. Set the username and password.
   Note: When you're asked to enter your password in Ubuntu, nothing will appear on the screen-not even asterisks (*). This is done for security reasons. Just type your password as usual, even though you can't see it, and press Enter.

```
Launching Ubuntu-24.04...
Provisioning the new WSL instance Ubuntu-24.04
This might take a while...
Create a default Unix user account: bebito
New password:
```

5. Now that you have installed the Ubuntu OS, update the system software:
   $: sudo apt update
   $: sudo apt upgrade
   When asked to confirm upgrading, type "Y" and press Enter.

```
Need to get 155 MB of archives.
After this operation, 149 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
```

6. Validate the Ubuntu version:
   $: lsb_release -a

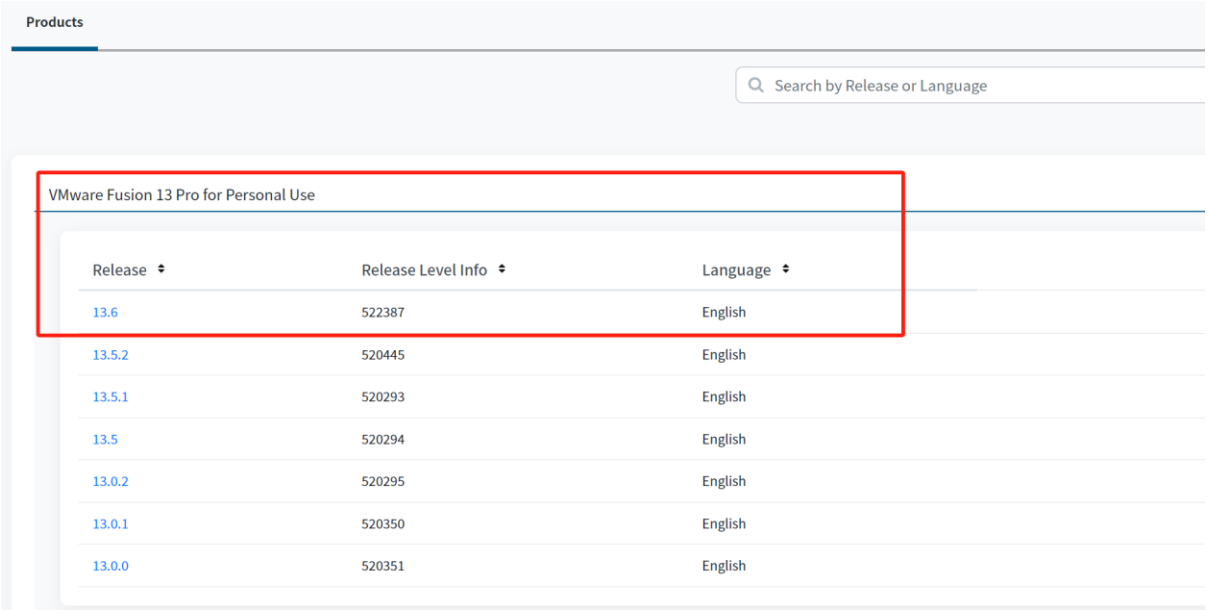
```
bebito@LAPTOP-FLFKRVEF:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.3 LTS
Release:        24.04
Codename:       noble
```

Install Ubuntu on MacOS (If you have a Mac computer)

If you do not have access to an Ubuntu or Windows machine, you may install Ubuntu on MacOS.

Installing Ubuntu on MacOS requires a virtual machine. Here are the steps:

1. Install the virtual machine: VMware Fusion.
   a. Register an account at Broadcom.
      https://profile.broadcom.com/web/registration
   b. Download the VMware Fusion.
      https://support.broadcom.com/group/ecx/productdownloads?subfamily=VMware+Fusion



2. Install Ubuntu on VMware Fusion. You can refer to the following link: https://www.youtube.com/watch?v=zDgSwcAOSq8

   Note:

   a. When following online tutorials to install Ubuntu, please make sure to choose the recommended version, **Ubuntu 22.04**.

   b. When searching for tutorials, remember to include your CPU type, either Intel series or M series, in the keywords. This will help you find the correct tutorial.

II. Installing ROS 2 Jazzy on Ubuntu

1. Enter the Ubuntu system. Here we give two ways to open the Ubuntu system:
   - Click the Windows Start menu, type "WSL" in the search bar, and when the related WSL option appears in the application list, click on it to open the WSL Ubuntu environment.
   - Click the Windows Start menu, locate the "Ubuntu 24.04.3 LTS" icon in the application list, and click on it to launch the Ubuntu environment in WSL.

2. Follow the steps below to install ROS 2. One of the ROS 2 versions corresponding to Ubuntu 24.04 is Jazzy.
   Note: If you use copy and paste, please note the Ubuntu terminal doesn't support "Ctrl+v" for pasting commands, but you can simply right-click in the terminal to paste.

   a. Update Ubuntu packages (good hygiene step).
   $: sudo apt update
   $: sudo apt -y upgrade

   b. Install prerequisites (tools needed to add ROS safely).
   $: sudo apt install -y curl gnupg lsb-release ca-certificates
   Also, enable the Ubuntu "universe" repo (ROS packages often need it):
   $: sudo apt install -y software-properties-common
   $: sudo add-apt-repository universe
   $: sudo apt update

   c. Add the ROS 2 "keyring" (security step).
   Ubuntu won't trust software sources unless you add their signing key.
   $: sudo mkdir -p /etc/apt/keyrings
   $: curl -fsSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /tmp/ros.key
   $: sudo gpg --dearmor -o /etc/apt/keyrings/ros-archive-keyring.gpg /tmp/ros.key

   d. Add the ROS 2 repository (so apt can find ROS).
   $: echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

e. Refresh apt again (so it reads the new ROS source).
$: sudo apt update

f. Install ROS 2 Jazzy.
$: sudo apt install -y ros-jazzy-desktop

g. Install "developer tools"
When you create packages, you'll compile code, resolve dependencies, and build whole workspaces. Ubuntu doesn't include the ROS-specific build tools by default, so we install them here:
$: sudo apt install -y python3-rosdep python3-colcon-common-extensions python3-vcstool build-essential python3-argcomplete

h. Initialize rosdep
Installing rosdep isn't enough — you must "activate" it once so it can use its database of dependency rules.
$: sudo rosdep init
$: rosdep update

i. Enable ROS commands every time you open a terminal.
Right now, ROS commands (like ros2) may work only in the current terminal unless you "source" ROS.
Sourcing loads ROS environment variables (paths) so your terminal can find ROS executables and packages.

- One-time check (works only for this terminal):
  $: source /opt/ros/jazzy/setup.bash
  This enables ROS only in the terminal you are using now. If you open a new terminal, you would need to source again—unless you make it automatic.

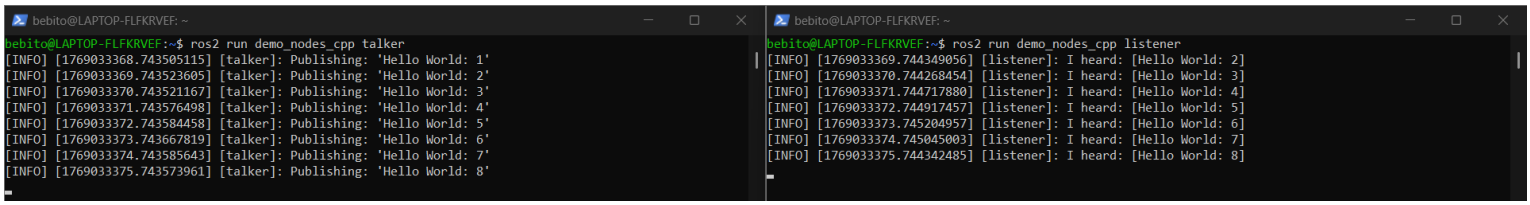
- Make it automatic:
  $: echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
  $: source ~/.bashrc
  The first line adds ROS sourcing to ~/.bashrc (a file that runs automatically when a new terminal opens).
  The second line reloads .bashrc so ROS is enabled immediately in the current terminal too.

j. Validate the ROS version.
$: echo $ROS_DISTRO

```
bebito@LAPTOP-FLFKRVEF:~$ echo $ROS_DISTRO
jazzy
```

k. Quick proof test: talker/listener demo.
Open two Ubuntu terminals (two WSL windows).

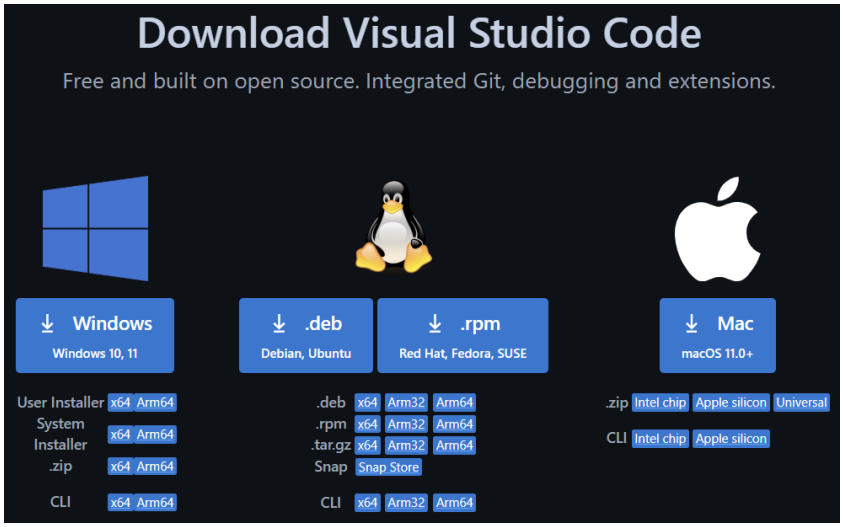
- Terminal 1:
  $: ros2 run demo_nodes_cpp talker
- Terminal 2:
  $: ros2 run demo_nodes_cpp listener



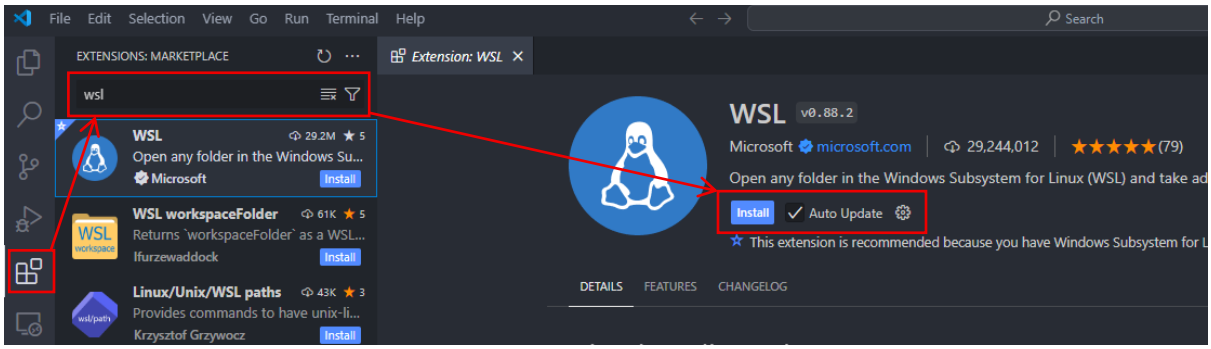
Expected: The listener prints messages like “I heard: [Hello World: #]” repeatedly.

III. Setting up VS Code on Windows to connect to WSL-Ubuntu

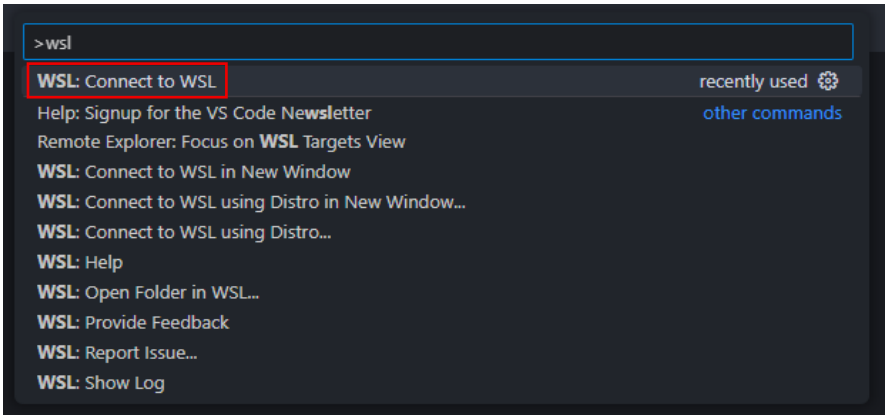
1. Use the following link to download VSCode based on your Operation System: https://code.visualstudio.com/Download



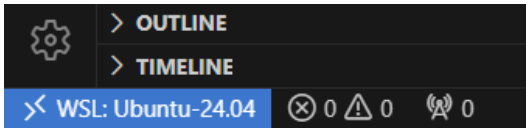
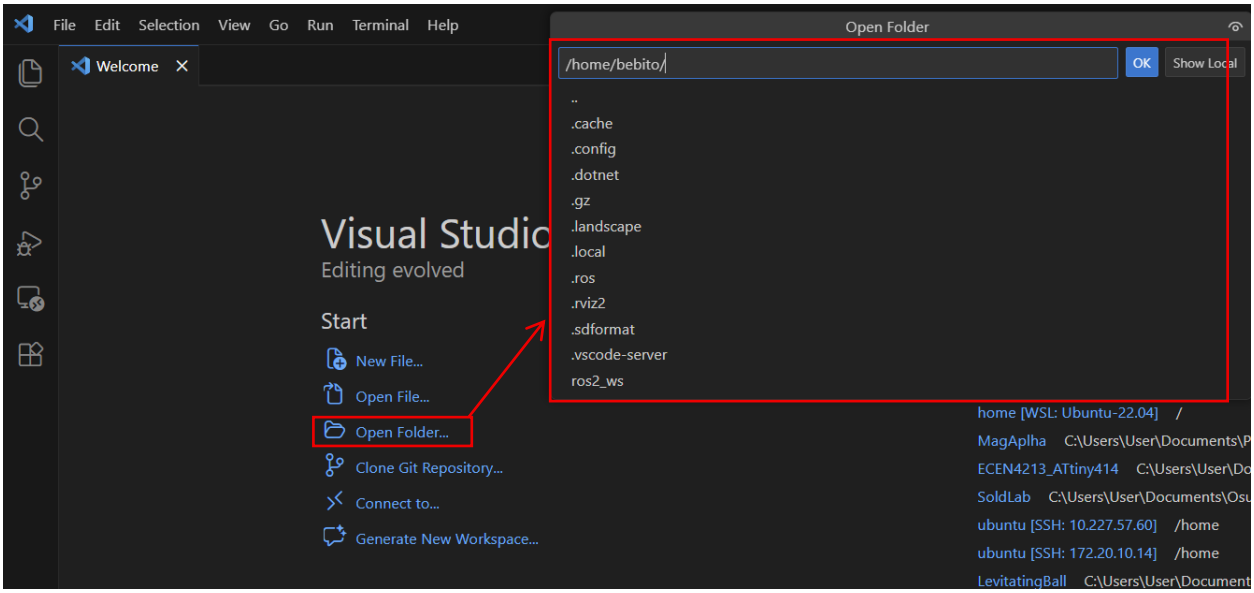
2. Search for "WSL" in Extensions search bar, then click the Install button.



3. Press "Ctrl+Shift+p" to open the command palette, then type "WSL". You should see the option to connect to WSL. Click on it to establish the connection.



4. After connecting to WSL, the status bar in the bottom left corner of VSCode should display the connection status.



5. Click Open Folder in the left toolbar to access the Ubuntu file system. From there, you can manage the system files using VSCode's GUI and start developing your program.